

AIM

To write a C program to implement Stack and Queue using Linked List, and to perform stack operations such as push, pop, checking empty and full conditions, and queue operations such as insertion and deletion.

PROCEDURE

1. Start the program.
2. Define a structure for the linked list node containing data and next pointer.
3. Implement stack using linked list by maintaining a top pointer.
4. Create functions to check whether the stack is empty or full.
5. Implement stack push operation by inserting a node at the beginning.
6. Implement stack pop operation by deleting a node from the beginning.
7. Implement queue using linked list by maintaining front and rear pointers.
8. Perform queue push (enqueue) operation by inserting a node at the rear.
9. Perform queue pop (dequeue) operation by deleting a node from the front.
10. Display appropriate messages for each operation.
11. Stop the program.

CODE

```
/*
File name   : sq.c
Author      : Monish Kanna
Purpose     : To implement Stack and Queue using Linked List. Stack operations include push, pop,
              and checking empty or full conditions. Queue operations include insertion (push) and
              deletion (pop).
*/

#include <stdio.h>
#include <stdlib.h>
struct Node
{
    int data;
    struct Node *next;
};
struct Node *top = NULL;
int isEmpty()
{ return (top == NULL); }
int isStackFull()
{
    struct Node *temp = (struct Node*)malloc(sizeof(struct Node));
    if (temp == NULL)
        return 1;
    free(temp);
    return 0;
}
void pushStack(int value)
{
    if (isStackFull())
    {
        printf("Stack Overflow\n");
        return;
    }
```

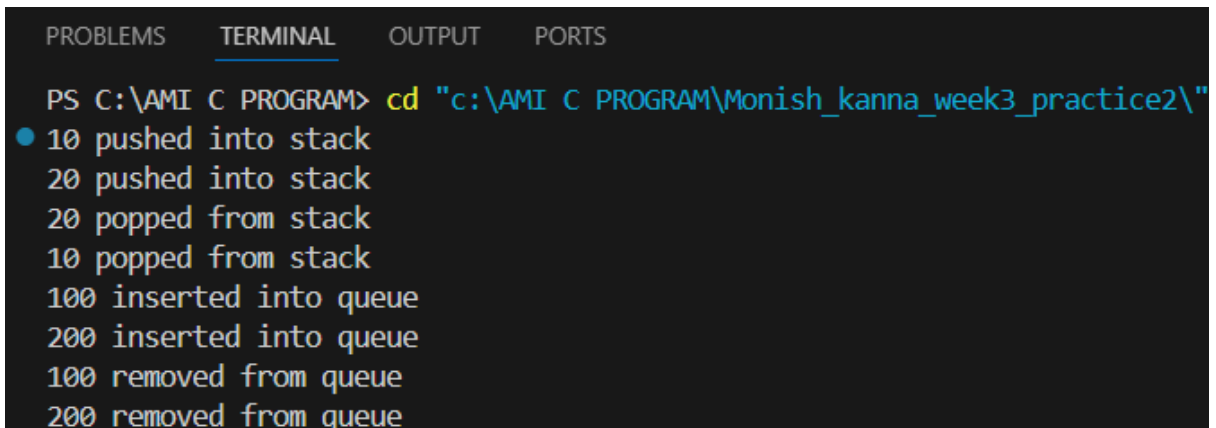
```

    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    printf("%d pushed into stack\n", value);
}
void popStack()
{
    if (isStackEmpty())
    {
        printf("Stack Underflow\n");
        return;
    }
    struct Node *temp = top;
    printf("%d popped from stack\n", temp->data);
    top = top->next;
    free(temp);
}
struct Node *front = NULL;
struct Node *rear = NULL;
void pushQueue(int value)
{
    struct Node *newNode = (struct Node*)malloc(sizeof(struct Node));
    newNode->data = value;
    newNode->next = NULL;
    if (rear == NULL)
    { front = rear = newNode; }
    else
    {
        rear->next = newNode;
        rear = newNode;
    }
    printf("%d inserted into queue\n", value);
}
void popQueue()
{
    if (front == NULL)
    {
        printf("Queue is Empty\n");
        return;
    }
    struct Node *temp = front;
    printf("%d removed from queue\n", temp->data);
    front = front->next;
    if (front == NULL)
        rear = NULL;
    free(temp);
}

```

```
int main()
{
    pushStack(10);
    pushStack(20);
    popStack();
    popStack();
    pushQueue(100);
    pushQueue(200);
    popQueue();
    popQueue();
    return 0;
}
```

OUTPUT



```
PROBLEMS  TERMINAL  OUTPUT  PORTS

PS C:\AMI C PROGRAM> cd "c:\AMI C PROGRAM\Monish_kanna_week3_practice2\"
● 10 pushed into stack
  20 pushed into stack
  20 popped from stack
  10 popped from stack
  100 inserted into queue
  200 inserted into queue
  100 removed from queue
  200 removed from queue
```

RESULT

The C program to implement Stack and Queue using Linked List was executed successfully. Stack operations (push, pop, empty and full check) and Queue operations (insertion and deletion) were performed correctly using linked list concepts.